

Management de Project Logiciels

Document complet - Chapitres 1 a 9

Genere le : 11/05/2026

Mode : images



Figure 1.1 – Domaine des connaissances en Management en Génie Logiciel (SWEBOOK.2023)

Les logiciels sont des constructions logiques exprimées sous forme algorithmique, contrairement à des produits physiques qui sont réalisés dans les domaines du génie mécanique, électrique, chimique, biologique et autres. Le logiciel est ainsi perçu comme immatériel, car il n'a pas de propriétés physiques palpables par ses utilisateurs contrairement à un objet physique. Il est donc considéré plus malléable en raison de la relative facilité avec laquelle le code source d'un logiciel peut être modifié. Le matériel est différent du logiciel. Le matériel est rarement modifiable et adaptable. Un logiciel est plus durable car il peut évoluer et être continuellement amélioré dans une organisation. Les personnes (surtout les administrateurs) qui ne connaissent pas le domaine font souvent pression pour obtenir l'effet souhaité en demandant des modifications rapides au code du logiciel, en pensant que ces modifications de code, en soi, sont simples par rapport à la modification d'éléments physiques d'un objet après sa construction. Mais les ingénieurs logiciels savent qu'à mesure que les logiciels et les systèmes intégrés deviennent de plus en plus gros, complexes, critiques et imbriqués, l'architecture et l'ingénierie logicielle ainsi que les rôles joués par les intervenants deviennent très importants.

Plusieurs spécialistes disent aussi qu'il devrait être possible de contrôler un projet logiciel et TI comme n'importe quel autre projet (c.-à-d. à l'aide des meilleures pratiques du PMI), mais il y a des aspects spécifiques au domaine des technologies de l'information et aux modèles de cycle de vie de développement logiciel qui rendent plus compliqué un management prévisible et complètement planifiable à l'avance – comme : La perception des clients qui ont souvent un manque d'appréciation pour la complexité inhérente du domaine, en particulier par rapport à l'impact des changements dans les exigences et les technologies de l'information en général. Il est presque inévitable que les processus et les technologies elles-mêmes, du génie logiciel et TI, mènent à de nouvelles exigences ou à des changements pour les clients. En conséquence, les projets logiciels et TI utilisent de plus en plus un processus itératif plutôt que les approches en cascade qui forcent une planification détaillée et complète de tâches planifiées en début du projet.

En ce qui concerne les ingénieurs du domaine du logiciel et TI, les activités de management de projet se produisent à trois niveaux : 1) organisationnel, 2) infrastructures TI, et 3) projets de développement/acquisition de logiciels. Les aspects du management organisationnel sont importants en matière de leur impact

sur le chef de projet (ou le Scrum master) — par exemple sur le management des politiques. Les politiques et les normes organisationnelles fournissent le cadre dans lequel le projet logiciel est mis en œuvre. Ces politiques peuvent être influencées par des exigences de développement/maintenance et opération des infrastructures TI efficaces, et un certain nombre de politiques spécifiques au domaine peuvent devoir être établies pour la gouvernance efficace des technologies de l'information à un niveau organisationnel. La figure 1.1 décrit l'ensemble du domaine des connaissances en management qui sont nécessaires à l'ingénieur logiciel pour bien gérer ses projets.

Par exemple, nous verrons que des politiques sont habituellement nécessaires pour établir des processus ou des procédures spécifiques pour l'ensemble de l'organisation pour des tâches de projets logiciels et TI telles que la conception, le développement/l'acquisition, l'estimation, l'infrastructure et le suivi des projets. De telles politiques sont essentielles pour un management à long terme efficace des technologies de l'information, en établissant une base cohérente pour analyser la performance antérieure et pour implanter des améliorations, par exemple. Un autre aspect important du management est le management du personnel : les politiques et les procédures d'embauche, la formation, la motivation et le mentorat pour le développement de la vie professionnelle sont importants non seulement au niveau des projets, mais également pour le succès à plus long terme d'une organisation. Le personnel du domaine du logiciel et des technologies de l'information peut représenter des défis uniques de formation et de management du personnel (par exemple, toujours s'assurer de la pertinence des compétences dans un contexte où la technologie sous-jacente subit un changement continu et rapide).

Le management des communications est également souvent négligé alors qu'il s'agit d'une problématique fondamentale pour les individus dans un domaine où la compréhension précise des besoins d'utilisateur ainsi que des exigences et des conceptions complexes est nécessaire. Finalement, pour appuyer la gestion de projet, le management de portefeuilles de projets, à l'aide d'un bureau de projet, permet d'avoir une vision globale non seulement de l'ensemble des projets logiciels en cours de développement, mais également des changements aux logiciels déjà en service dans une organisation.

Au tout début d'un projet, il est important de se faire une idée de sa portée et d'estimer son coût. L'échec et le coût sont des demandes initiales des gestionnaires, par exemple – est-ce que vous pensez que ce projet pourrait durer six semaines et avoir un budget d'environ 20 000 \$? Pour répondre à cette question, il faut identifier la portée du projet, qui fait référence à la liste des produits livrables et des fonctionnalités entendues avec le client. C'est à ce stade que l'ampleur de la solution requise peut être planifiée avec un peu plus de précision. Par exemple, la création d'un nouveau site Web pour l'entreprise peut être réalisable en six semaines, mais réécrire le logiciel de comptabilité ne l'est pas. Nous allons voir dans ce livre que Carl Weigers d'Eastman Kodak (Weigers, 1996), a proposé une approche (voir figure 1.2) afin de mieux encadrer les discussions de démarrage de projets de développement logiciel. Il propose cinq dimensions qui doivent être négociées tout au début d'un projet logiciel.

cultures, de contextes, de structures, de responsabilités, de désirs et d'objectifs différents.

Pourtant, des objectifs semblables peuvent être identifiés à chaque étape du développement d'un logiciel. Il n'y a, en fait, pas beaucoup de variations sur la façon dont les travaux sont décrits, organisés et gérés. Par conséquent, les cycles de vie de développement de logiciels modernes sont suffisamment flexibles pour être utilisés dans différents types d'activités, de produits et de services. La liste suivante présente des modèles/méthodologies de cycle de vie qui sont utilisés dans l'industrie du logiciel depuis 1960 :

- Cascade,
- Prototypage rapide (RAD),
- Incémentiel,
- Processus Unifié (UP) - (c.-à-d. à la fois itératif et incémentiel),
- Cycle de vie pour les projets d'intelligence artificielle,
- Cycle de vie agile.

La figure 1.4, tirée du PMBOK, présente la synthèse des caractéristiques d'un projet selon qu'il utilise un cycle de vie plus classique (c.-à-d. prédictif) ou plus agile. Notez qu'il y a un lien entre le choix de cycle de vie et la cadence de livraison. Les cycles de vies plus classiques ont tendance à livrer une seule fois ou quelques fois tandis que les cycles de vie agiles livrent périodiquement ou continuellement. Cette figure présente les différentes façons de traiter les exigences et les plans, la gestion des risques et des coûts, les considérations liées à l'échéancier et la gestion de l'implication des principales parties prenantes en fonction du type de cycle de vie utilisé. Les cycles de vie d'un projet prédictif se caractérisent par l'attention qu'ils portent à la spécification des exigences et une planification détaillée au cours des premières étapes du projet. Des plans détaillés fondés sur les exigences et les contraintes connues peuvent réduire les risques et les coûts. Des jalons pour la participation des principales parties prenantes sont également prévus. À mesure que le plan détaillé est exécuté, les processus de maîtrise visent à limiter les changements susceptibles d'influer sur le périmètre, l'échéancier ou le budget.

Cycle prédictif	Cycle itératif	Cycle incémentiel	Cycle agile
Les exigences sont définies au départ avant le début du développement.	Les exigences peuvent être définies à intervalles réguliers au cours de la livraison.	Les exigences sont définies fréquemment en cours de livraison.	
Des plans sont proposés pour le livrable attendu. Ensuite, une fois un produit final est livré à l'échéance du projet.	La livraison peut être livrée en sous-ensembles du produit total, global intéressant pour le client.	La livraison a lieu fréquemment avec des sous-ensembles du produit global intéressant pour le client.	
Le changement est limité autant que possible.	Tout changement est intégré à intervalles périodiques.	Tout changement est intégré en temps réel lors de la livraison.	
Les principales parties prenantes interviennent à des jalons spécifiques.	Les principales parties prenantes interviennent régulièrement.	Les principales parties prenantes interviennent continuellement.	
Les risques et les coûts sont maîtrisés à l'aide de la planification détaillée de considérations susceptibles d'être connues.	Les risques et les coûts sont maîtrisés grâce à l'élaboration progressive des plans sur la base des nouvelles informations.	Les risques et les coûts sont maîtrisés au fur et à mesure que les exigences et les contraintes apparaissent.	

Figure 1.4 – Le continuum des cycles de vie du projet (PMBOK, 2017 - Figure X3-1)

Les cycles de vie très itératifs, incémentiels et agiles se caractérisent par l'élaboration progressive des exigences sur la base de petits cycles de planification et d'exécution itératifs. Les risques et les coûts sont réduits grâce à l'évolution progressive des plans initiaux. Les principales parties prenantes sont impliquées en permanence et donnent des retours d'information fréquents, ce qui permet de réagir aux changements plus rapidement et de garantir une meilleure qualité. Les considérations suivantes s'appliquent au centre du continuum de ces cycles de vie : les risques et les coûts sont réduits grâce à l'évolution itérative des plans initiaux, les principales parties prenantes ont plus d'opportunités de participer aux cycles incémentiels, itératifs et agiles que lors des jalons du projet dans les cycles de vie très prédictifs. Les cycles de vie du projet situés au centre du continuum du cycle de vie (voir la figure 1.4) ont tendance à mieux refléter le côté prédictif ou le côté agile selon la façon dont les exigences sont spécifiées, la gestion des risques et des coûts mais aussi la nature de la participation des principales parties prenantes. Les projets dans cette partie du continuum peuvent utiliser des méthodes de projet hybrides.

Ainsi vous devez identifier si votre projet comporte un degré important d'incertitude au niveau des exigences et de l'approche à sélectionner afin de les satisfaire. Ces incertitudes peuvent contribuer à des taux de changement élevés et au niveau de la complexité du projet. Ces caractéristiques sont illustrées à la figure 1.5. L'incertitude du projet augmente avec le risque de reprise du livrable et le besoin d'utiliser un cycle de vie différent. Pour limiter l'impact de ces risques, les équipes sélectionnent des cycles de vie qui leur permettent de gérer le projet comportant une incertitude élevée au moyen de petits incréments de travail, c'est donc l'approche agile qui est choisie. Elles peuvent vérifier l'avancement du travail lorsqu'elles utilisent de petits incréments et modifier ce qui vient ensuite. Lorsque les équipes livrent de petits incréments, elles peuvent mieux comprendre les réelles exigences du client, de façon plus rapide et précise qu'avec une spécification écrite statique.

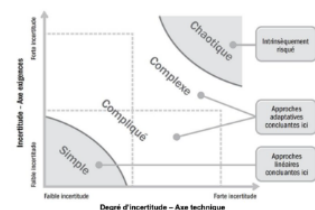


Figure 1.5 – Modèle d'incertitude et de complexité (PMI - Figure 2.5)

Il convient de souligner que les cycles de vie de développement logiciel sont complexes et multidimensionnels. Souvent, les multiples étapes d'un projet donné utilisent différents cycles de vie, tout comme des projets distincts au sein d'un programme.

particulier peuvent être exécutés à l'aide de différents cycles de vie. Voyons les particularités de ces cycles de vie de développement logiciel.

1.3.1 LE CYCLE DE VIE EN CASCADE (PRÉDICTIFS)

Le modèle de cycle de vie en cascade est la méthodologie de développement logiciel la plus ancienne, la plus connue et encore populaire dans certains domaines d'affaires. Dans un cycle de vie prédictif, le périmètre, la durée et les coûts du projet sont déterminés au cours des premières phases du cycle de vie. Les changements apportés au périmètre du projet doivent être rigoureusement gérés. Les cycles de vie prédictifs peuvent aussi être appelés cycles de vie en cascades. C'est un cycle de vie séquentiel facile à comprendre et à utiliser. C'est un cycle de vie prédictif, dans le sens que chaque étape doit être terminée avant de commencer l'étape suivante, ce qui signifie qu'aucun chevauchement n'est autorisé. La sortie de chaque étape sert d'entrée pour la prochaine étape. Une illustration du modèle de cascade se trouve ci-dessous :

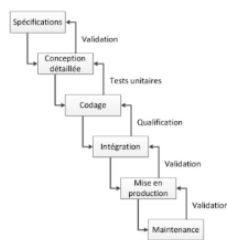


Figure 1.6 – Diagramme générique d'un modèle de cycle de vie en Cascade

Faciles à comprendre, surtout pour le personnel inexpérimenté, ses étapes sont bien définies et les exigences/spécifications doivent être claires et approuvées avant de passer à une autre étape. Cependant, dans cette approche, toutes les exigences doivent être connues avant de démarrer. Ainsi les livrables sont fixés et il devient difficile d'accepter des demandes de changements sans avoir à tout reprendre depuis le début. Cette approche est critiquée pour être longue avant de pouvoir démontrer de l'avancement au client, car il ne pourra voir le résultat du travail qu'à l'étape juste avant la mise en production. Ainsi, ce cycle de vie a été beaucoup critiqué pour son effet de « Big Bang » lors de la livraison finale et aussi parce que les clients ne participent pas assez pendant toute la durée de la conception, du codage et des essais d'intégration et système du logiciel.

1.3.2 LE CYCLE DE VIE DE PROTOTYPAGE RAPIDE (RAD)

Pour pallier le manque d'interaction avec les clients, et pour les situations où la clientèle ne sait pas ce qu'elle veut, ce cycle de vie propose d'effectuer un prototype jetable (par opposition à un prototype évolutif) qui a pour seul but de confirmer rapidement avec le client ses exigences détaillées. C'est donc le début de l'ajout d'une étape itérative à une approche prédictive. Ensuite, quand le prototype d'interface est précisé, un cycle de vie en cascade est utilisé pour développer le logiciel.

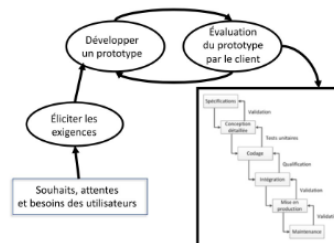


Figure 1.7 – Exemple d'un cycle de vie de prototypage rapide

Les difficultés de ce cycle de vie sont associées aux deux questions suivantes : 1) quelle technologie est la plus efficace pour effectuer un prototype jetable rapidement et quelle technologie sera utilisée pour développer le logiciel ? Certains langages de programmation sont plus appropriés pour le prototypage que pour l'implémentation, certains plus appropriés pour l'implémentation que le prototypage, et certains sont appropriés pour les deux. 2) combien d'itérations de prototypage permettront de découvrir tous les besoins de l'utilisateur (c.-à-d. pas trop ou pas trop peu) ?

Avec ce cycle de vie, quand le client s'impatiente et demande à voir le prototype, cela indique aux développeurs qu'il est peut-être temps d'arrêter de prototyper.

1.3.3 LE CYCLE DE VIE INCRÉMENTIEL

Dans le cas d'un cycle de vie incrémentiel, les livrables proviennent d'une série d'itérations qui ajoutent progressivement des fonctionnalités dans une période prédéterminée. Les livrables incluent les fonctionnalités nécessaires et suffisantes pour être considérées comme exhaustives uniquement après l'itération finale. Ce modèle est une évolution du modèle en cascade. Il débute tout comme un cycle de vie en cascade pour quelques étapes de démarrage : les exigences, l'architecture et une partie de la conception (des fonctions les plus complexes/risquées). Par la suite, chaque cycle de livraison développe un produit fonctionnel ou le client peut donner son opinion.

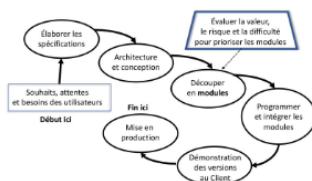


Figure 1.8 – Diagramme générique d'un modèle de cycle de vie incrémental

Chaque incrément subséquent demande de la conception additionnelle et aussi du travail d'intégration (aussi nommé *refactoring*). Ce cycle de vie demande de diviser le projet en modules à l'aide de certains critères d'évaluation (c.-à-d. le risque, la difficulté, et la valeur pour le client). Il permet une rétroaction client plus fréquente que le modèle en cascade.

1.3.4 LE CYCLE DE VIE DU PROCESSUS UNIFIÉ (ITÉRATIF)

Ce cycle de vie se caractérise par une démarche itérative, incrémentale et introduit des notions d'agilité pilotée par des cas d'utilisation regroupés en étapes, et centrée sur l'architecture et les modèles UML. Dans ce cycle de vie, plusieurs itérations pouvaient être démarrées en parallèle. Ce processus intégrait toutes les activités de conception et de réalisation au sein de cycles de développement composé de 4 activités itératives : la conception, l'élaboration, la construction et la transition. Ce cycle de vie logiciel n'était pas un processus prescriptif concret unique, mais plutôt un cadre de processus adaptable, destiné à être adapté par les organisations de développement et les équipes de projet logiciel qui sélectionneront les éléments du processus adaptés à leurs besoins. Nous présentons ici RUP qui était une implémentation spécifique du Processus Unifié (UP). RUP était un cadre de processus de développement logiciel itératif créé par Rational Software Corporation, une division d'IBM de 2003 à 2008. L'enjeu de ce cycle de vie était de former et d'utiliser tous les outils Rational d'IBM avant de démarrer le projet. Les difficultés de gestion de ce cycle de vie étaient associées à : 1) un choix judicieux d'un sous-ensemble de cas d'utilisation (c.-à-d. pas trop ou pas trop peu) qui est effectué à l'aide d'une analyse de priorités/risques de chaque cas d'utilisation, 2) combien d'itérations doit-on effectuer (c.-à-d. pas trop ou pas trop peu), et 3) assigner les bonnes personnes aux bonnes tâches pour obtenir les meilleurs résultats.

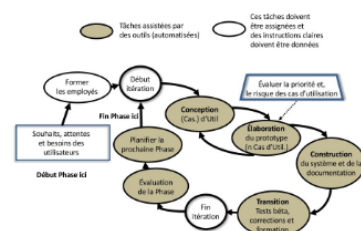


Figure 1.9 – Diagramme générique d'un modèle de cycle de vie du Processus Unifié de Rational

L'identification des risques majeurs, la mise en place d'une architecture cible du logiciel à concevoir et la planification des étapes d'élaboration seront les principales tâches à effectuer. Ce cycle de vie mettait l'accent sur une architecture/conception tôt lors d'une itération exprimée sous forme de modèles UML, suivie d'un prototypage pour éclaircir les besoins. Au terme de ces deux activités, l'activité de construction est le moment où l'on construit le logiciel et on le documente. La fin d'une itération, la transition, le logiciel est en version bêta en production et un groupe d'utilisateurs essaie le logiciel et détecte les anomalies et défauts. L'évaluation de l'étape actuelle permet d'évaluer si l'on a fait trop ou trop peu durant l'itération et donne des informations pour la planification du nombre d'itérations pour la prochaine étape.

1.3.5 LE CYCLE DE VIE DE L'INTELLIGENCE ARTIFICIELLE (IA)

Nous aimerions présenter un dernier cycle de vie de développement logiciel, celui pour les projets d'intelligence artificielle. Peu importe si l'apprenant est un humain ou une machine, le processus d'apprentissage de base est similaire. Il peut être divisé en six étapes :

1. **L'abstraction** implique l'identification d'un contexte d'affaires et d'un objectif à atteindre ou d'une question à répondre,
2. **La compréhension des données** vise à s'assurer que chaque élément de donnée soit compris (c.-à-d. quel est le sens de la donnée) afin d'atteindre l'objectif,
3. **La préparation des données** vise à ce que les données disponibles soient traitées et organisées en préparation à leur utilisation. Il est souvent nécessaire de retravailler les données de manière qu'elles soient utilisables par les algorithmes d'apprentissage machine,
4. **La modélisation** consiste à utiliser un/des algorithmes d'intelligence artificielle sur un sous-ensemble de données pour en concevoir un modèle,
5. **L'évaluation** utilise le modèle sur un autre sous-ensemble de données (souvent plus grand) et fournit ainsi un mécanisme de rétroaction pour mesurer l'utilité des connaissances acquises et informer les améliorations potentielles au modèle, et

6. Le déploiement où finalement quand le modèle est validé il peut être inséré dans un logiciel de production et ainsi ses prédictions/décisions seront utilisées.

La figure 1.10 illustre les étapes d'un processus itératif typique de mise en œuvre d'un modèle d'intelligence artificielle pour solutionner un problème d'affaires :

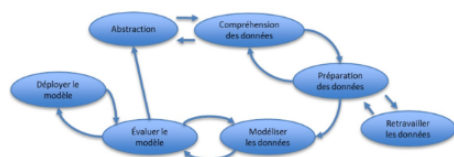


Figure 1.10 – Diagramme générique d'un modèle de cycle de vie d'intelligence artificielle

1.3.6 LES CYCLES DE VIE (MÉTHODES) AGILES

Le terme « méthode agile » est parfois trop réducteur pour parler de ce populaire cycle de vie de gestion de projet logiciel et TI. Il s'agit de bien plus qu'une panoplie de méthodes. On parle plutôt de paradigme agile, d'état d'esprit agile, de philosophie agile, de culture agile ou encore d'approche agile, de mouvement agile, de courant agile.

Les méthodes agiles tentent de remédier aux effets négatifs des modèles cycles de vie dits « traditionnels ». Mais comment fonctionne le management d'un projet agile moderne ? Il repose sur un principe simple, mettre l'accent sur le pourquoi. C'est-à-dire donner du sens plutôt que des directives. C'est ce qui fait la différence entre un gestionnaire de projet et un leader. Le gestionnaire de projet classique utilise le pouvoir de l'autorité pour dire aux employés ce qu'ils doivent faire et le leur imposer. Alors qu'un leader utilise le pouvoir de conviction et de motivation interne pour accompagner les personnes vers leurs objectifs. Adopter une démarche agile c'est savoir que l'on risque de changer de plan voire d'objectifs plusieurs fois au cours du projet. Laisser de l'autonomie à son équipe cela veut dire accepter qu'elle ne se comporte pas d'une façon préétablie et ne suit pas une planification détaillée préétablie. C'est aussi savoir qu'un leader agile a le pouvoir d'appuyer les initiatives même lorsqu'elles sortent du cadre. Ainsi il doit donc respecter les individualités, développer les compétences de chacun, favoriser l'auto-organisation et se dire qu'il anime l'équipe autant que les individus.

Les méthodes agiles partent du principe que spécifier et planifier, dans les détails, l'intégralité d'un produit avant de le développer (c.-à-d. utiliser l'approche itérative) est contreproductif. Cela revient à planifier dans les détails un trajet « Montréal – Miami » en voiture par les routes secondaires. Spécifiant chaque ville et village traversés, l'heure de passage associée, chaque rue empruntée dans les quartiers, les litres d'essence consommés, kilomètres parcourus, etc. Les imprévus ne manqueront pas d'arriver : embouteillages, déviations, travaux, sens de circulation inversés, intempéries, etc., rendant cette planification détaillée et les spécifications du voyage

très vite obsolètes. Combien de temps aurez-vous passé à planifier cet itinéraire, comment réagirez-vous face à vos frustrations de ne pas pouvoir appliquer votre plan à la lettre ?

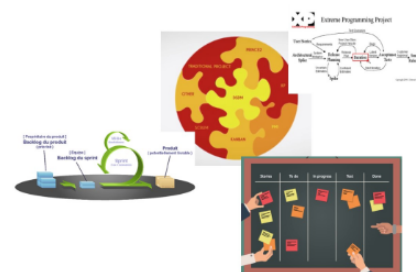


Figure 1.11 – Représentation générique d'un modèle de cycle de vie agile

En agilité, l'idée consiste à se fixer un premier objectif à court terme (une grande ville par exemple) et se lancer sur la route sans tarder. Une fois ce premier objectif atteint, on marque une courte pause et on adapte son itinéraire en fonction de la situation du moment. Et ainsi de suite jusqu'à atteindre Miami. On parle donc d'une approche légère qui s'adapte au moment. Dans le cadre d'un projet de développement logiciel, le client élabore sa vision du logiciel à réaliser et liste les fonctionnalités et les exigences. Il soumet cette liste à l'équipe de développement, communique directement avec elle (plutôt que par papier) qui estime l'effort de chaque élément de la liste. On peut ainsi se faire une idée approximative du budget global.

L'équipe sélectionne ensuite une portion des exigences à réaliser dans une portion de temps court appelée itération. Chaque itération inclut des travaux de conception, de spécification fonctionnelle et technique quand c'est nécessaire, de développement et de test. À la fin de chacune de ces itérations, le produit partiel, mais utilisable est montré au client. Ce dernier peut alors se rendre compte par lui-même très tôt du travail réalisé, de l'alignement sur le besoin. L'utilisateur final quant à lui peut se projeter dans l'usage du produit et émettre des rétroactions précieuses pour les futures itérations. La visibilité ainsi offerte est clé. Cette transparence peut également apporter davantage de confiance et de collaboration dans la relation client/fournisseur. Les risques quant à eux sont levés très tôt.

Si le client a priorisé avec soin son besoin, il peut saisir l'opportunité d'accélérer le projet s'il estime que le produit en état partiel peut aller en production, en économisant ainsi son budget et en récoltant un premier retour sur investissement. Il a aussi la possibilité de changer, en cours de route, la priorité des fonctionnalités qui n'ont pas encore été développées (c.-à-d. prévues pour les itérations futures), afin de retarder une fonctionnalité dont le besoin n'est pas critique, ou même ajouter une nouvelle

fonctionnalité plus importante en échange du retrait d'une autre (respectant ainsi budget et délais).

Mais comme tout modèle de cycle de vie, l'approche agile comporte ses problématiques et défis propres, car elle souffre autant des demandes clients qui insistent sur des délais de livraison fixes et invariables avant de débiter un projet. Celui-ci peut tourner le projet en une course contre la montre, qui bien souvent met en péril la qualité du produit. Il arrive souvent que le budget fixé pour un projet soit trop faible ou peu réaliste, parce qu'éloigné d'une compréhension détaillée des activités de développement logiciel. Ce décalage a pour conséquence non seulement de ralentir l'avancée du projet, mais également d'impacter sa qualité. Il faut anticiper le coût de l'agilité. La nécessité de plusieurs rencontres Scrum sont critiquées, car elles prennent un temps précieux. Par exemple, la réunion « stand up » de 15 minutes (c.-à-d. la mêlée quotidienne) qui se déroule chaque matin et qui parfois s'étend jusqu'à 30 minutes. Multiplié par le nombre de développeurs et de jours cela représente beaucoup de temps. Par exemple, pour une équipe de 8 personnes, cela représente donc un effort entre 20 et 40 heures sur un Sprint de deux semaines, soit l'équivalent d'une personne à mi-temps. Dès le départ, il faut en être conscient. Je donne l'exemple du « stand-up », mais les autres réunions ou discussions ne prennent pas moins d'effort, bien au contraire. Une autre problématique est qu'il faut faire attention de ne pas trop délaissier des tâches techniques importantes de *refactoring*, par exemple, pour faire entrer une tâche de développement avec l'objectif de faire plaisir au client ou au chef d'un autre département. Finalement il faut toujours se protéger quand un client croit que si ça marche en démo, ça marchera en production ce soir. Le client ne sait pas que la démonstration est souvent effectuée avant d'avoir complété tous les tests, parce que malheureusement, certaines équipes accumulent une « dette technique », soit des activités de contrôle de qualité non effectuées alors qu'elles présentent les fonctionnalités comme étant terminées, mais elles ne le sont pas vraiment. Bref, ces équipes escamotent l'approche agile.

En conclusion, il appartient à l'équipe de management de projet de déterminer le meilleur cycle de vie pour chaque projet. Le cycle de vie du projet doit être suffisamment flexible pour traiter les divers facteurs du projet. Cette flexibilité peut être acquise en :

- Identifiant le ou les processus à réaliser pour chacune des phases,
- Réalisant le ou les processus identifiés dans la phase correspondante, et
- Adaptant les divers attributs d'une phase (par exemple, son contenu, sa durée, ses critères de sortie et ses critères d'entrée).

L'enjeu du chef de projet ou du Scrum master est de faire prendre conscience des enjeux des cinq dimensions présentés à la figure 1.2) d'un projet logiciel à tous les intervenants dès le début du projet. Le cycle de vie du projet est distinct du cycle de vie du produit (qui peut être développé par un projet). Le cycle de vie du produit est une série de phases qui représentent l'évolution d'un produit, du concept, en passant par la livraison, la croissance et la maturité jusqu'à son retrait du marché.

1.4 L'IMPACT DU CHOIX DE MODÈLE SUR LA PLANIFICATION ET LA GESTION DE PROJET

Le choix du cycle de vie du projet dans le domaine des technologies de l'information a un impact sur l'approche de gestion de projet qui est préférée par l'organisation. Les approches traditionnelles de planification de projet, avec un plan de projet formel, sont un style de gestion de projet qui nécessite que les exigences et la planification soient claires et bien comprises par tous les intervenants et puissent être décrites en détail avant de démarrer un projet logiciel. Cette approche de gestion de projet logiciel, nécessitant un plan de projet détaillé avant de démarrer, s'avère moins populaire parce que le niveau de détail n'est souvent pas disponible au début. Aujourd'hui, avec l'accélération de la compétition entre les entreprises, il est pénalisant pour l'entreprise de démarrer un projet seulement quand son plan précis sera développé en détail et approuvé préalablement. Certaines grandes entreprises utilisent encore cette approche de gestion de projet et demandent à leurs chefs de projets de produire et faire approuver des plans détaillés avant d'autoriser le lancement d'un projet logiciel. Il a été observé que les organisations qui utilisent les cycles de vie en cascade, incrémentale, et RUP utilisaient plus souvent cette approche de planification de projets.

Alternativement nous avons tous constaté l'émergence d'un style plus itératif de gestion de projet qui permet de démarrer le projet logiciel même si le plan de projet n'est pas détaillé ni précis. Cette approche reconnaît que les exigences ne sont pas encore claires et qu'il est possible de démarrer et progressivement d'y voir clair pendant son exécution. Le terme itératif est aussi un terme qui n'est pas défini clairement, car c'est assez rare qu'un projet logiciel en démarre sans aucun plan. Nous verrons le détail de cette approche dans le prochain chapitre.

Ainsi, ce qui est important, c'est que ces termes de gestion de projet logiciel « par plan » ou « itératif » ne soient pas perçus comme des décisions mutuellement exclusives. Vous aurez toujours à utiliser différentes approches de gestion de projet logiciel comportant des modèles cycle de vie variés (c.-à-d. cascade, itératif, prototypage, etc.) tout au long de votre carrière qui comporteront différents niveaux de planification préalable.

Il faut aussi tenir compte de ce que votre employeur demande à ce titre. Par exemple, la certification en gestion de projet « par plan » nécessite, selon le PMI version 7 publié en 2021, les connaissances des 3 domaines : 1) les individus, 2) les processus, et 3) l'environnement d'affaires qui étaient disponibles dans la version antérieure du guide et que progressivement les questions vont évoluer cette année pour se concentrer sur le contenu du guide version 7 (30% à 40% des questions). L'examen de certification 2022 comporte 280 questions en près de quatre heures (230 minutes):

- **Les individus** (42% des questions de l'examen PMI-PMP®) : Les sujets abordés sont : la gestion de conflits, comment diriger une équipe, comment appuyer la performance de l'équipe, responsabiliser l'équipe et les parties prenantes, former l'équipe et les parties prenantes, construire une équipe, négocier les ententes, collaborer et encadrer avec les parties prenantes, développer une compréhension partagée, engager et soutenir les équipes virtuelles, définir les règles d'engagement de l'équipe, utiliser et évaluer l'intelligence émotionnelle des coéquipiers,
- **Les processus** (50% des questions de l'examen PMI-PMP®) : Les sujets abordés sont : l'exécution du projet selon l'urgence requise afin d'offrir une valeur commerciale, gérer les communications, gérer les risques, engager les parties prenantes, planifier et gérer le budget, les ressources, les échéanciers, la qualité des produits/livrables,

la portée du projet. Intégrer l'ensemble des activités et gérer les changements, les problèmes et les aspects d'acquisition. Identifier une méthodologie appropriée et gérer les artefacts. S'assurer de la continuité à la suite de l'implémentation et la clôture de projet.

- L'environnement d'affaires (8% des questions de l'examen PMI-PMP®) : Les sujets abordés sont : la planification et la gestion de la conformité, l'évaluation du retour sur investissement et de la récolte des bénéfices escomptés, l'évaluation des changements organisationnels et de l'environnement externe sur la portée du projet, appuyer les changements organisationnels.

Dire qu'un cycle de vie agile est meilleur qu'un cycle de vie en cascade revient à dire qu'une automobile est mieux qu'un bateau. Ces deux approches de cycle de vie ont été conçues pour développer du logiciel selon des philosophies différentes et pendant votre carrière il y en aura bien d'autres de proposées et vous allez devoir vous y adapter. Donc, je suis d'avis qu'il est essentiel que l'ingénieur en logiciel et en TI soit capable objectivement de comprendre les différents modèles de cycle de vie logiciels et d'adapter leur approche de gestion de projet selon le cas. Il est certain qu'en 2022 la profession de gestion de projet logiciel, dans le domaine des technologies de l'information, subit des changements profonds en ce moment. Il y a plusieurs facteurs qui forcent ces changements :

- La nature des projets logiciels change : la gestion de projet classique a été développée pour de grands projets d'infrastructure et de logiciels. De nos jours les projets de types : développement Web, commerce électronique et applications mobiles ne requièrent pas autant de planification.
- Les technologies de développement logiciel et d'infrastructures TI changent : la popularité grandissante des services infonuagiques, des outils de DevOps et des « cadriciels » de développement accélèrent grandement le rythme des projets logiciels tout en éliminant de plus en plus les étapes à franchir avant de livrer en production.

Ce genre de situation, où il y a des changements drastiquement différents dans la façon d'effectuer une activité, se nomme un changement de paradigme. Un changement similaire a eu lieu dans les années 2000 avec le mouvement de la qualité qui a révolutionné le domaine de fabrication des automobiles et qui a passé d'une approche de contrôle de la qualité vers des approches de qualité totale. Ces changements de paradigmes, dans le domaine des technologies de l'information, arrivent assez fréquemment et les ingénieurs logiciels doivent s'habituer à étreindre ces changements. Les prochaines sections introduisent les concepts du deuxième thème de ce livre : l'assurance qualité logicielle.

1.5 EXERCICES DE SIMULATION DE CYCLES DE VIE LOGICIELS

L'objectif de ces simulations est de découvrir l'impact de vos décisions concernant différentes approches de gestion de projet selon le cycle de vie logiciel que vous décidez d'utiliser. Conséquemment, vous allez comprendre que, pour le gestionnaire de projet logiciel, le choix d'un cycle de vie a un grand impact sur sa manière de gérer son projet. Donc, prenez le temps de vous documenter, lire et comprendre, à l'aide des références suivantes, le fonctionnement particulier de chaque cycle de vie de développement logiciel. Ainsi en vous documentant sur son fonctionnement spécifique vous allez mieux comprendre comment avoir du succès dans le jeu de simulation pour chaque cas de figure.

Références pour le cycle de vie en Cascade :

1. Boehm, B.W., *Software Engineering Economics*. 1981, Upper Saddle River, NJ: Prentice Hall, Inc.
2. Brooks, F.P., *The Mythical Man-Month: Essays on Software Engineering*. 2 ed. 1974, Boston, MA: Addison-Wesley. 336.
3. Bryan, G.E., *Not All Programmers are Created Equal*, in *Software Engineering Project Management*, R.H. Thayer, Editor. 1997, IEEE Computer Society: Los Alamitos, CA. p. 346-355.
4. Curtis, B., H. Krasner, and N. Iscoe, *A Field Study of the Software Design Process for Large Systems*. Communications of the ACM, 1998. **31**(11): p. 1268-1287.
5. Sackman, H., W.J. Erikson, and E.E. Grant, *Exploratory Experimental Studies Comparing Online and Offline Programming Performance*. Communications of the ACM, 1968. **11**(1): p. 3-11.

Références pour le cycle de vie incrémental :

1. Boehm, B.W., *A Spiral Model of Software Development and Enhancement*. IEEE Computer, 1988. **21**(5): p. 61-72.
2. Gilb, T., *Evolutionary Delivery versus the Waterfall Model*. ACM SIGSOFT Software Engineering Notes, 1985: p. 49-61.
3. Larman, C. and V. Basili, *Iterative and Incremental Development: A Brief History*. IEEE Computer, 2003. **36**(6): p. 47-56.
4. Royce, W., *TRW's Ada Process Model for Incremental Development of Large Software Systems*, in *Proceedings of the 12th International Conference on Software Engineering*. 1990. p. 2-11.
5. Scacchi, W., *Process Models in Software Engineering*, in *Encyclopedia of Software Engineering*, J. Marciniak, Editor. 2001, Wiley.

Références pour le cycle de vie de prototypage rapide (RAD) :

1. Gordon, V.S., Bieman, J.M. *A Rapid prototyping: Lessons Learned*. IEEE Software, 1995. **12**(1): p. 85-95.
2. Kordon, F., Luqi. *An introduction to rapid system prototyping*. IEEE Transactions on Software Engineering, 2002. **28**(9): p. 817-821.
3. Gomaa, H. and Yusuf, Y.Y., *The impact of Rapid prototyping on user requirements*. ACM SIGSOFT, 1983.
4. Hartson, H.R. and Smith, E.C. *Rapid prototyping in human computer interface development*, *Interacting with Computers*, 1991. **3**(1): p. 51-91.

Références pour le cycle de vie RUP :

1. Kruchten, P. *The Rational Unified Process: An Introduction*, (3rd Edition). 2000: Addison-Wesley.

2. Kruchten, P. *Going over the waterfall with the RUP*, Rational Edge Magazine, 2001, 16 p.
3. Ambler, S.W. *A manager's Introduction to Rational Unified process (RUP)*. AmbySoft, 2005, 18 p.
4. Smith, J. *A comparison of RUP and XP*, Rational Software, White Paper, 2002, 22 p.
5. Parnas, D., and Clements, P.C. *A Rational Design process: How and why to fake it*, IEEE Transactions, SE-12(2), February 1986.

Références pour le cycle de vie Extreme Programming (XP) :

1. Cockburn, A. and Williams, L. *The Costs and Benefits of Pair Programming, in Extreme Programming Examined*. 2000, Addison Wesley.
2. Wake, W.C., *Extreme Programming Explained*. 2002, Boston, MA: Addison-Wesley.
3. Juric, R. *Extreme Programming and its development practices*, Proceedings of the 22nd International Conference on Information Technology Interfaces, 13-16 Juin 2000, p. 97-104.
4. Ji, F. and Sedano, T. *Comparing extreme programming and waterfall project results*, Proceedings of the 24th IEEE-CS Conference on Software Engineering Education and Training (CSE&T), 22-24 Mai 2011, p. 482-486.
5. Beck, K. *Embracing change with extreme programming*, Computer, 32(10), Oct 1999: p. 70-77.

Installer SIMSE sur votre poste de travail :

Allez à l'adresse web <https://www.ics.uci.edu/~emiljo/SimSE/> et trouvez la section « Downloads ».

Pour MacOS :

1. Vérifiez que vous avez le *Java SE Development kit 14: Java jdk-14-osx-x64-bin* d'installé pour votre MacOS. Sur Mac, la dernière version de Java fonctionne bien avec le jeu.
2. Téléchargez les fichiers (.jar) de chacun des 4 cycles de vie dans la section « Download » du site SIMSE : **Games** = (waterfall, incremental, rapid prototyping, Rational Unified Process et Extreme Programming) et mettez-les sur votre bureau.
3. Démarrez un terminal :
 1. Changez de répertoire avec la commande « cd desktop ».
 2. Démarrez un jeu avec la commande « java -jar waterfall.jar ».
 3. Changez **waterfall** par le nom des autres cycles de vie pour les démarrer.

Pour Windows 10 :

- 1- Les dernières versions de Java ne fonctionnent pas bien avec ce jeu. Vérifiez que vous avez le Java SE 11 (LTS) Java jdk-11.0.6_windows-x64_bin.exe (le Windows x64 Installer) d'installé sur votre équipement. Si vous avez une version plus récente de Java, enlevez-la et remplacez-la par la version 11.0.6 (vous pourrez la remettre après le cours).
- 2- Téléchargez les fichiers (.jar) de chacun des 4 cycles de vie dans la section « Download » du site SIMSE : **Games** = (waterfall, incremental, rapid prototyping, Rational Unified Process et Extreme Programming) sur votre bureau.
- 3- Double-cliquez sur le fichier .jar et le jeu débute.

Comprendre le fonctionnement du jeu :

- 1- Débutez avec le tutoriel du simulateur **SIMSE** : <https://www.youtube.com/watch?v=kjphP/rj6Pv&list=PL3g>
- 2- Formez-vous sur l'outil de branchement SIMSE pour répéter une partie de la simulation à partir d'un point précis et prendre une autre décision : <https://www.youtube.com/watch?v=t-g2huTfmQ&list=PL3g>
- 3- Étudiez l'outil explicatif SIMSE qui permet de révéler des détails de la simulation : <https://www.youtube.com/watch?v=iiDjmiB9-6U&list=PL3g>

Prenez et conservez des notes pour l'examen en ligne :

Pour ce Travail pratique, vous devez effectuer la simulation de tous les cycles de vie et prendre des notes personnelles qui vous seront utiles pour répondre aux questions lors de l'examen :

- Notez votre meilleure note obtenue et le nombre de « ticks » (c.-à-d. jours) et notez le nombre maximum de « ticks » à ne pas dépasser pour obtenir le meilleur score.
- Pour tous les cycles de vie, notez la séquence des étapes (stratégie) à suivre pour obtenir le meilleur résultat.
- Pour tous les cycles de vie, retenez le nom du système à développer.
- Notez l'impact de l'assignation des personnes aux tâches selon leurs compétences/expérience et d'impliquer les bonnes personnes au bon moment.
- Notez l'importance des revues pour découvrir les défauts qui se cachent dans le logiciel (tentez de voir, avec l'outil explicatif combien de défauts se cachent dans votre projet).
- Notez l'impact d'acheter ou pas des outils d'ingénierie.
- Analysez les objets (par exemple: l'énergie, le « Mood » et le « Payrate » des employés, les caractéristiques des livrables (Nb d'erreurs détectées, Nb d'erreurs non détectées, %d'erreur, %complété) et l'impact de l'utilisation d'outils de génie logiciel (Coût, facteur d'augmentation de la productivité et facteur de réduction d'erreurs).

Pour le cycle de vie waterfall :

- a) Quel est l'effet de congédier André dès le début du projet ?
- b) Qu'est qu'un débutant comme Roger peut et ne peut pas faire comme tâche ?

- c) Qu'arrive-t-il si vous ne terminez pas complètement une étape et démarrez quand même l'étape suivante ?
- d) Qu'arrive-t-il (2 impacts) quand on met trop d'employés au travail sur une même tâche ?
- e) À quel moment il est préférable d'effectuer la revue d'un artefact ?
- f) Quel est l'impact de ne pas préparer les tests avant de les effectuer ?
- g) Comment doit-on traiter une demande de changement efficacement dans ce cycle de vie ?

Pour le cycle de vie incrémental :

- a) Quel est l'effet de sauter une ou plusieurs étapes de la documentation (exigences ou conception) sur un ou plusieurs modules ?
- b) Quels sont les attributs les plus importants pour prendre des décisions ?
- c) Quel attribut s'améliore si on effectue les exigences avant la conception ?
- d) Essayez de sauter l'étape des exigences pour voir l'effet.
- e) Quel est l'impact d'une soumission précoce ?

Pour le cycle de vie prototypage rapide :

- a) Essayez tous les langages et identifiez le bon langage pour prototyper versus l'implémentation finale. Et aussi regardez l'effet de l'utilisation du même langage pour ces deux activités.
- b) Quel est l'effet de faire trop peu ou beaucoup trop de prototypage sur le projet ?

Pour le cycle de vie RUP :

- a) Combien de fois pouvez-vous acheter des outils ?
- b) Quels sont les artefacts produits lors de l'« inception » ?
- c) Quelles sont les activités principales de l'étape de développement ?
- d) Quel est l'effet de faire trop ou pas assez d'itérations dans une étape ?
- e) Quelle étape prend le plus de « ticks » ?

Pour le cycle de vie Extreme Programming :

- a) Quelles sont les activités d'une réunion de planification ?
- b) Que se passe-t-il si vous n'incluez pas le client dans les réunions ?
- c) Que se passe-t-il si vous n'incluez pas le développeur dans une réunion de planification ?
- d) Que se passe-t-il si vous ne faites pas les tests unitaires au bon moment ?
- e) À quel moment faut-il faire les tests pour avoir le meilleur score ?
- f) Est-ce que le budget est important dans ce jeu ?

1.6 SOMMAIRE DES APPRENTISSAGES DU CHAPITRE 1

Voici les connaissances à bien réviser pour avoir les meilleures chances de succès à l'examen :

- Rappelez-vous de la définition d'un projet,
- A quel endroit pouvez-vous retrouver les connaissances de l'ingénieur logiciel en management de projet logiciel ?
- Rappelez-vous pourquoi c'est difficile de gérer des projets logiciels et sa notion d'intangibilité,
- Expliquez pourquoi un manager qui ne connaît pas le domaine du logiciel pourrait mettre une pression indue sur l'équipe pour avoir des résultats ?

- Rappelez-vous des 5 dimensions à négocier dès le départ avec vos clients et vos patrons,
- Soyez capable de lister et expliquer la différence entre les différents modèles de cycles de vie logiciel : au niveau de la gestion des exigences, la livraison, le changement, les parties prenantes et les risques/coûts,
- Soyez capable de débattre des avantages et inconvénients de l'utilisation de l'agile,
- Assurez-vous de répondre aux questions pour chaque cycle de vie (SIMSE), par exemple : Identifiez la séquence d'étapes qui donne le meilleur résultat pour le Waterfall ?
 - 1) Laisser 1 ou 2 des meilleurs employés à la correction des erreurs qui ont été détectées dans les exigences,
 - 2) Débuter la création de la prochaine étape avec la même stratégie. Il y aura forcément un peu d'employés disponibles pour la correction des artefacts précédents, mais l'idée reste la même,
 - 3) À la suite de la collecte des exigences, intégrer tous les employés à la revue des exigences,
 - 4) Assigner les 3 ou 4 meilleurs employés au travail,
 - 5) Lors de nouvelles exigences, il faut respecter la cascade en partant de la première étape, soit les exigences, et répéter la même boucle définie à partir du début : Choix de réponses : a)2,4,3,2,1, **b)4,3,1,2,5**, c)4,3,2,5,1, d)4,3,2,5,4, e)1,3,4,2,5
- Un autre exemple de question : Vrai ou Faux. Dans le cycle de vie Extreme Programming, est-ce vrai qu'un développeur qui ne participe pas à la réunion de planification ne pourra pas participer à l'itération suivante ? Réponse : Vrai.

1

Introduction au management de projets logiciels

Dans ce chapitre, les sujets suivants sont abordés :

- Le domaine de connaissances en management pour l'ingénieur logiciel (SWEBOK),
- Les différents modèles de cycle de vie de développement logiciel,
- L'impact du choix d'un modèle sur la planification et le management de projet,
- La qualité dans le domaine logiciel et TI,
- Connaître la différence entre une faute, une erreur et une défaillance et d'où proviennent-elles,
- Les perspectives de la qualité logicielle,
- Introduire les sujets abordés dans le domaine de l'assurance qualité logicielle,
- Introduire un logiciel de simulation des cycles de vie du logiciel (SIMSE).

1.1 INTRODUCTION

Nous sommes tous passés par là : le projet semble bien se dérouler, mais si vous êtes tout à fait honnête, vous êtes probablement un peu en retard. Vous surveillez mentalement toutes ces petites tâches dont vous avez besoin pour vous assurer de terminer à temps. Ensuite, le client appelle avec des modifications. Une semaine plus tard, vous redoutez l'appel « comment ça va », car vous savez que vous n'avez plus aucune idée du temps que cela va prendre. Vous êtes perdu au milieu de toutes les demandes qui s'accumulent. Vous avez besoin de management de projet.

Dans ce chapitre, les notions de différents cycles de vie de développement logiciel et leurs caractéristiques les plus importantes sont présentées pour le gestionnaire de projet. Il sera aussi question de la raison pourquoi le management de projet a tendance à être un sujet que beaucoup d'ingénieurs logiciels trouvent moins passionnant et conséquemment pourquoi les *compétences cognitives* en management de projet sont de plus en plus en demande aujourd'hui. Ensuite, nous discuterons de ce que le management de projet n'est pas et verrons à quel point une mauvaise utilisation des techniques et outils peut entraîner des complications et des retards.

1.2 COMMENT DÉFINIR LE MANAGEMENT DE PROJET POUR L'INGÉNIEUR LOGICIEL ET TI

Une définition officielle du management de projet, gracieuseté du PMI - *Project Management Institute*, définit le terme comme suit : « application des connaissances, des compétences, des outils et des techniques aux activités du projet pour répondre aux exigences du projet ».

Une description plus tangible serait que le management de projet est tout ce dont vous avez besoin pour réaliser un projet dans les délais impartis et dans les limites du budget, afin de fournir la portée et la qualité requises. Un projet classique se distingue du travail habituel en ce qu'il s'agit d'un effort ponctuel assez important qu'il requière une équipe assemblée pour le faire. Par exemple, la création d'un nouveau site Web serait donc un projet. Cependant, sa maintenance quotidienne et les mises à jour mineures ne le seraient pas.

La gestion, en génie logiciel, peut être définie comme un ensemble d'activités de travail impliquées dans la planification, l'estimation, la mesure, le contrôle, la coordination, la direction et la gestion des facteurs de risque pour superviser un projet de logiciel afin de garantir que les produits logiciels sortants et les services d'ingénierie logicielle sont fournis de manière efficiente, efficace et au profit des parties prenantes. Ces notions sont présentées plus en détail au chapitre 9 du guide [SWEBOK](#) (version 2023 à venir) car il est en cours de mise à jour au moment d'écrire ce livre. Dans cette nouvelle version du SWEBOK on insiste beaucoup plus qu'avant sur ces notions (voir tous les sujets abordés à la figure 1.1 qui suit).

Dans un sens, il devrait être possible de gérer un projet de génie logiciel de la même manière que d'autres projets complexes. Cependant, les ingénieurs logiciels utilisent différents modèles de processus, techniques et styles de résolution de problèmes qui sont fondés sur leurs normes, leurs connaissances et leurs expériences et aussi prennent en compte les différences entre les attributs qualité requis, selon la criticité du logiciel. Il faut aussi considérer que le logiciel n'est pas un objet physique.

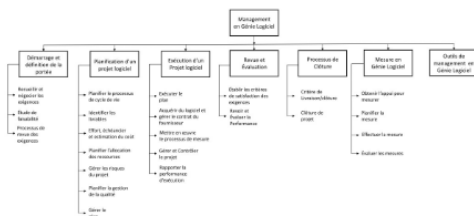


Figure 1.1 – Domaine des connaissances en Management en Génie Logiciel (SWEBOK, 2023)

Les logiciels sont des constructions logiques exprimées sous forme algorithmique, contrairement à des produits physiques qui sont réalisés dans les domaines du génie mécanique, électrique, chimique, biologique et autres. Le logiciel est ainsi perçu comme immatériel, car il n'a pas de propriétés physiques palpables par ses utilisateurs contrairement à un objet physique. Il est donc considéré plus malléable en raison de la relative facilité avec laquelle le code source d'un logiciel peut être modifié. Le matériel est différent du logiciel. Le matériel est rarement modifiable et adaptable. Un logiciel est plus durable car il peut évoluer et être continuellement amélioré dans une organisation. Les personnes (surtout les administrateurs) qui ne connaissent pas le domaine font souvent pression pour obtenir l'effet souhaité en demandant des modifications rapides au code du logiciel, en pensant que ces modifications de code, en soi, sont simples par rapport à la modification d'éléments physiques d'un objet après sa construction. Mais les ingénieurs logiciels savent qu'à mesure que les logiciels et les systèmes intégrés deviennent de plus en plus gros, complexes, critiques et imbriqués, l'architecture et l'ingénierie logicielles ainsi que les rôles joués par les intervenants deviennent très importants.

Plusieurs spécialistes disent aussi qu'il devrait être possible de contrôler un projet logiciel et TI comme n'importe quel autre projet (c.-à-d. à l'aide des meilleures pratiques du PMI), mais il y a des aspects spécifiques au domaine des technologies de l'information et aux modèles de cycle de vie de développement logiciel qui rendent plus compliqué un management prévisible et complètement planifiable à l'avance – comme : La perception des clients qui ont souvent un manque d'appréciation pour la complexité inhérente du domaine, en particulier par rapport à l'impact des changements dans les exigences et les technologies de l'information en général. Il est presque inévitable que les processus et les technologies elles-mêmes, du génie logiciel et TI, mènent à de nouvelles exigences ou à des changements pour les clients. En conséquence, les projets logiciels et TI utilisent de plus en plus un processus itératif plutôt que les approches en cascades qui forcent une planification détaillée et complète de tâches planifiées en début du projet.

En ce qui concerne les ingénieurs du domaine du logiciel et TI, les activités de management de projet se produisent à trois niveaux : 1) organisationnel, 2) infrastructures TI, et 3) projets de développement/acquisition de logiciels. Les aspects du management organisationnel sont importants en matière de leur impact

sur le chef de projet (ou le Scrum master) — par exemple sur le management des politiques. Les politiques et les normes organisationnelles fournissent le cadre dans lequel le projet logiciel est mis en œuvre. Ces politiques peuvent être influencées par des exigences de développement/maintenance et opération des infrastructures TI efficaces, et un certain nombre de politiques spécifiques au domaine peuvent devoir être établies pour la gouvernance efficace des technologies de l'information à un niveau organisationnel. La figure 1.1 décrit l'ensemble du domaine des connaissances en management qui sont nécessaires à l'ingénieur logiciel pour bien gérer ses projets.

Par exemple, nous verrons que des politiques sont habituellement nécessaires pour établir des processus ou des procédures spécifiques pour l'ensemble de l'organisation pour des tâches de projets logiciels et TI telles que la conception, le développement/l'acquisition, l'estimation, l'infrastructure et le suivi des projets. De telles politiques sont essentielles pour un management à long terme efficace des technologies de l'information, en établissant une base cohérente pour analyser la performance antérieure et pour implanter des améliorations, par exemple. Un autre aspect important du management est le management du personnel : les politiques et les procédures d'embauche, la formation, la motivation et le mentorat pour le développement de la vie professionnelle sont importants non seulement au niveau des projets, mais également pour le succès à plus long terme d'une organisation. Le personnel du domaine du logiciel et des technologies de l'information peut représenter des défis uniques de formation et de management du personnel (par exemple, toujours s'assurer de la pertinence des compétences dans un contexte où la technologie sous-jacente subit un changement continu et rapide).

Le management des communications est également souvent négligé alors qu'il s'agit d'une problématique fondamentale pour les individus dans un domaine où la compréhension précise des besoins d'utilisateur ainsi que des exigences et des conceptions complexes est nécessaire. Finalement, pour appuyer la gestion de projet, le management de portefeuilles de projets, à l'aide d'un bureau de projet, permet d'avoir une vision globale non seulement de l'ensemble des projets logiciels en cours de développement, mais également des changements aux logiciels déjà en service dans une organisation.

Au tout début d'un projet, il est important de se faire une idée de sa portée et d'estimer son coût. L'échec et le coût sont des demandes initiales des gestionnaires, par exemple – est-ce que vous pensez que ce projet pourrait durer six semaines et avoir un budget d'environ 20 000 \$? Pour répondre à cette question, il faut identifier la portée du projet, qui fait référence à la liste des produits livrables et des fonctionnalités entendues avec le client. C'est à ce stade que l'ampleur de la solution requise peut être planifiée avec un peu plus de précision. Par exemple, la création d'un nouveau site Web pour l'entreprise peut être réalisable en six semaines, mais réécrire le logiciel de comptabilité ne l'est pas. Nous allons voir dans ce livre que Carl Weigers d'Eastman Kodak (Weigers, 1996), a proposé une approche (voir figure 1.2) afin de mieux encadrer les discussions de démarrage de projets de développement logiciel. Il propose cinq dimensions qui doivent être négociées tout au début d'un projet logiciel.



Figure 1.2 – Diagramme de flexibilité d'un projet (Weigers, 1996)

Weigers précise que ces 5 dimensions ne sont pas toutes indépendantes, par exemple si vous ajoutez des membres du personnel à l'équipe de projet il est probable (mais pas toujours observé) de raccourcir l'échéance, mais, conséquemment, le coût du projet va augmenter. Un compromis souvent envisagé afin de rencontrer une date fixe est souvent de demander s'il sera possible d'écourter l'échéance en diminuant les fonctionnalités. Ces compromis ne sont pas simples, mais ils peuvent être imagés à l'aide de ce diagramme afin de rendre les discussions de démarrage plus réalistes et de bien documenter les décisions. Ainsi pour chacun de vos projets, vous devez vous faire une idée de quelles dimensions seront plus contraignantes et quelles autres vous offriront un degré de liberté d'action pour compenser les contraintes imposées.

Trop souvent dans une discussion écourtée nous avons tendance à ne parler que du budget et de l'échéance. C'est souvent cette attitude qui nous mène dans les dépassements et l'insatisfaction chronique de notre industrie. Il faut donc habituer nos dirigeants et notre clientèle à ne pas toujours avoir toutes les fonctionnalités demandées, sans défauts, livrées rapidement, par une petite équipe, et ce à un petit coût. Il y a aussi l'influence du type de cycle de vie de développement à considérer ici.

Cycle de vie (life cycle) : Évolution d'un système, produit, service, projet ou autre entité d'origine humaine de la conception à la retraite (ISO12207,2017).

Cycle de développement (development life cycle) : Processus de cycle de vie du logiciel qui comporte des activités d'analyse des besoins, de conception, de codage, d'intégration, de tests, d'installation et de soutien pour l'acceptation des produits logiciels (ISO90003,2018).

Comme vous le savez peut-être, il y a plusieurs modèles de cycles de vie, c'est-à-dire différentes approches de développement de logiciel, disponibles et utilisées par les entreprises. Le modèle de cycle de vie d'un projet logiciel générique est assez simple : vous démarrez d'abord le projet (c.-à-d. planifier le projet), puis vous réalisez le projet (via les étapes de planification, d'exécution et de contrôle, qui forment une boucle), et enfin vous finissez par les essais finaux et la stratégie de mise en production (la clôture). Ce processus générique, proposé par le PMI, est illustré à la figure 1.3.

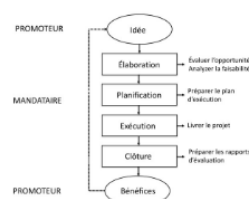


Figure 1.3 – Diagramme générique d'un modèle de cycle de vie d'un projet

1.3 LE CHOIX D'UN MODÈLE DE CYCLE DE VIE

Les premiers concepts de cycles de vie logiciels ont été créés dans les années 1960 pour développer des logiciels grande échelle. Au début de la programmation informatique, les seuls modèles utilisés pour développer des projets complexes étaient ceux de la construction et de la fabrication. Il était donc logique que les approches structurées utilisées dans ces industries soient également appliquées au développement de logiciels. Par exemple, dans le domaine de la construction, les architectes comprennent d'abord les besoins du client. Ils conçoivent ensuite une maquette pour le client qui accepte le concept. Ensuite les ingénieurs utilisent les plans et suivent des étapes pour développer et construire les bâtiments, les ponts ou les routes. À la dernière étape, ils inspectent le travail, le démontre au client et livrent le travail.

Dans les années 1970, la popularité de la programmation augmente et dans le domaine bancaire les grands ordinateurs sont utilisés. À cette époque le processus de développement traditionnel qui se déroule de manière séquentielle du début à la fin d'un projet est populaire. Les croisements entre les étapes principales d'un projet logiciel ne sont pas courants. C'est une époque où les techniques de programmation étaient complexes et les langages de programmation difficiles à apprendre et à manipuler. Ainsi, le développement d'un système informatique qui suit une approche structurée et séquentielle était assez épanouie. Au cours des 50 dernières années, les systèmes informatiques ont joué un rôle important d'amélioration de la productivité dans les entreprises. De l'envoi de courrier postal à l'envoi de courrier électronique via Internet, du remplissage de formulaires papier aux formulaires électroniques, du registre comptable manuel aux feuilles de calcul électroniques, chaque amélioration est étroitement liée à la mise en place d'une technologie de l'information. Par conséquent, de nombreuses entreprises prennent ces innovations très au sérieux et dépensent des sommes, des ressources et des efforts considérables en matière de technologie de l'information. En cinq décennies, les concepts de développement logiciel ont évolué et de nouvelles méthodologies de développement et de conception ont émergé pour accélérer la mise en œuvre d'applications et solutions orientées client. Chacun de ces cycles de vie de développement possède des avantages et des inconvénients, des forces et des faiblesses. Il n'est pas réaliste de penser qu'un seul cycle de vie logiciel puisse s'adapter à des millions d'organisations en raison de